

CAMP

Homework 4:

Report of Cache

Name : SeokBeom Lim (임석범)

Department : Mobile System Engineering

Student number : 32203743

Table of contents

1. 서론	3
2. 본론	
1) Cache 개념	3
2) Cache 의 Mapping과 Policy (해당 방식 채택 이유)	4
3) Cache 코드 설계	7
4) 결과값	12
5) 결과값을 통한 성능 비교 및 고찰	15
3. 결론	18

서론

컴퓨터 구조의 발전 역사에서, 여러 trade-off를 해결하고 더 나은 방법을 모색하여 컴퓨터 성능을 최적화 하였다. 컴퓨터 구조가 발전하는 과정에서 기존 시스템의 문제를 발견하고 이를 해결하고자 하는 여러 생각들이 혁신적인 아이디어를 창출 해냈다. 이러한 과정을 Von Neumann Arch, Single Cycle, Pipeline 까지 반복 하였다. 마지막 과제의 주제인 캐시는 앞선 기술들의 문제점(Von Neumann 병목현상, Memory 접근 속도와 Register의 접근 속도 차이)을 해결 할 수 있다. 이번 과제에서는 지난 과제에서 구현한 Pipeline Mips Simulator에 Cache Memory를 추가 하여 Memory Access 성능을 개선하는 방법에 대해 학습하고자 한다.

본론

I. Cache 개념

Pipeline Simulator를 코드로 구현하면서 Memory와 Register의 접근 속도에 대한 차이를 고려하지 않았었다. Memory 접근 속도는 CPU가 register에 접근하여 처리 하는 속도에 비해 접근하는데 시간이 오래 걸려서 상대적으로 느리다. 이에 따라, 현실적인 Pipeline 구조에서 CPU 처리 시간에 비해 Memory 처리 시간이 길어서 CPU가 memory 값이 처리되기 까지 waiting하는 상태가 된다. 이러한 문제점을 해결하기 위해 고안된 것이 Cache이다.

Cache는 컴퓨터 구조에서 Memory 접근 시간을 줄이기 위해 사용하는 Temporary Memory이다. Main memory 보다 작지만 빠른 접근 속도를 가진다. 이러한 Cache를 Main memory와 CPU 사이에 위치하게 하여 사용하는 데이터를 빠르게 접근하게 할 수 있다. 이때

Cache 는 접근하는 데이터를 효율적으로 Load, Store 하기 위해 두가지 Locality를 기준으로 데이터를 다룬다.

1. Temporal Locality

2. Spatial Locality

Temporal Locality는 “최근 접근한 데이터는 가까운 미래에 접근한다” 는 아이디어이며 시간적으로 가까운 데이터는 가까운 미래에 사용되는 경향이다.

Spatial Locality 는 “최근 접근한 주소의 인접한 주소의 값이 가까운 미래에 사용된다” 라는 아이디어를 가지며 공간적으로 가까운 데이터 (예를 들면, 배열과 같은)가 가까운 미래에 쓰이는 경향을 가진다.

Cache는 Cache Line(Cache Block) 으로 이루어져있으며, Cache Line은 Tag와 Data를 가진다. Tag는 Main memory에서 어떤 주소에서 왔는지 식별 하기 위해 사용 된다. Index 또한 Cache의 구성 요소중 하나이며 캐시에서의 캐시 라인의 위치를 나타내기 위해 주소에서 일부 비트를 사용한다.

II. Cache 의 Mapping과 Policy (해당 방식 채택 이유)

Cache는 여러가지 Mapping 방식이 있는데, Direct Mapped, Fully Associative Mapped, Set-Associative Mapped 방식이 있다. 이 중 가장 간단한 방법은 Direct Mapped 방식이다. Direct Mapped 방식은 Memory 주소를 각 Memory word의 캐시 내의 딱 한 곳에 Mapping 하는 방식이다. 이 방식의 장점은 간단하지만 같은 Tag 값을 가진 값이 들어 왔을 때 발생하는 충돌이 잦다. 이에 대비 되는 Fully Associative Mapped 방식은 모든 라인이 어떤 Memory의 블록에도 Mapping 할 수 있는 구조를 가진다. Fully Associative Mapped 방식은 블록이 어느

곳에나 있을 수 있기 때문에 Cache 내 모든 엔트리를 찾아 봐야 한다. 이에 따라 찾는 비용이 높아져, 실용적으로 사용 하기 위해서는 비교기를 사용하여 병렬적으로 검색을 해야 한다. 따라서, HW Price가 높다. 이에 따라 이 둘을 절충하여 사용하는 **Set Associative Mapped** 방식이 효율적이다. Set Associative Mapped 방식은 한 Cache Line에 들어 갈 수 있는 자리 개수가 고정 되어있다. 자리의 개수를 연관 정도로 부르는데 연관 정도는 주로 2,4,8,16, 과 같이 2의 제곱수가 사용된다. n-way Associative Mapped 에서 n이 연관 정도를 나타내는데, 여기서 trade-off 가 있는데, n의 값이 올라 갈 수록 Fully Associative Mapped 방식과 유사해지며 n의 값이 작아 질 수록 Directed Associative Mapped(= 1-way Associative Mapped) 와 유사해진다. 하지만, n의 값이 낮아 질수록 충돌이 잦아 지지만 n의 값이 너무 높으면 해당 목표 값을 찾는 데 적중 시간이 오래 걸린다. 따라서, **4-way Associative Mapped** 방식을 이번 코드에서 채택 하였다.

Cache 에서 데이터를 Write 할 때는 Read 할때 와 다르게 Cache에만 Write 하고 Memory에는 Write 하지 않으면 또는 이에 반대 상황이 와도, Cache 값과 Memory 값이 불일치 한다. 이를 일치 시킬 수 있는 간단한 방법은 Write-Through 이다. Write-Through 방식은 데이터의 일관성을 유지 할 수 있는 반면 Cache의 성능이 매우 낮아진다. 특히, Memory에 Write을 하는 경우, 시간이 Read에 비해 더욱 오래 걸린다. 이에 대한 해결책으로 두가지가 있다. 첫번째는 Write-Buffer 를 사용하는것이다. 하지만 매우 큰 프로그램을 돌리고 있을 때, Write buffer 가 가득 찬다면 결국 Write Buffer가 빌 때 까지 Stop이 발생 할 수 밖에 없다. 또한, Write 완성 속도와 Processor의 Write 속도차이가 난다면, 사이즈가 큰 Buffer 를 사용해도 도움이 되지 않는다. 따라서, Write Buffer를 사용하기 보다 두번째 해결 방법인 Write-Back 방법을 사용하면 Write-through의 문제를 완벽히 해결 할 수 있다. **Write Back** 방식은 새로운 값이 들어오면 Main Memory에 접근하여 직접 쓰는 것이 아닌, Cache 에만 쓰고 Cache에서 나갈 때 Main memory에 저장 되는 형식이다. Write Back 방식을 채택하게 되면, Cache Line에 변수 dirty가 추가 되어야 한다. dirty 비트는 해당 값이 Memory의 값과 일치 하는지의 여부를 나타낸다. 만약, Replacement Policy에 의해 값이 교체 된다고 할 때, 해당 dirty 값이 1이면 나가면서 Main memory에 저장이 되어야 한다. 만약 dirty의 값이 0이면 Main memory에 저장할

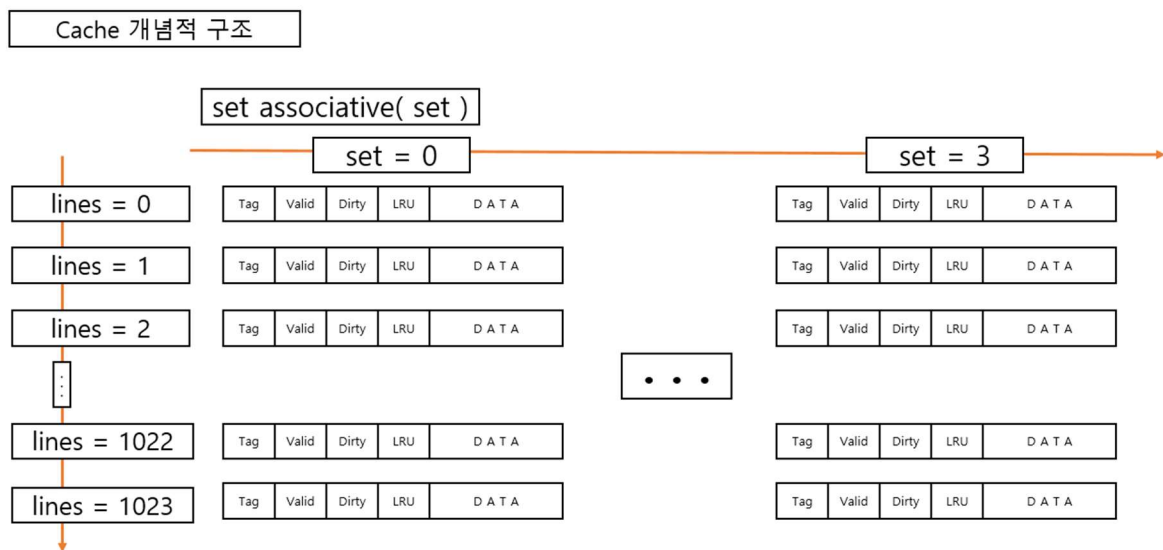
필요는 없다. 위와 같이, Write-Back 방식은 Cache의 성능을 다른 쓰기 방식보다 크게 높여줄 수 있다.

Cache 의 Set이 가득 찬 경우, 새로운 데이터를 교체해야 하는 Cache Replacement Policy 를 선택 하는 것 또한 Cache 의 성능을 올릴 수 있다. Cache Replacement Policy는 Cache set가 모두 차있는 경우, 해당 Cache set의 나가야 하는 Cache Line을 효율적으로 선택하는 Policy를 사용하여야 한다. 대표적으로, LRU, FIFO, RANDOM 이 Replacement Policy 선택사항중 하나로 꼽힌다. 이 중 Random은 항상 같은 성능 낼 수 없으므로 코드 설계에서 기각 되었다. FIFO 는 가장 먼저 들어온 데이터 값을 교체 하는것이다. FIFO Policy를 사용할 경우, 구현이 간단하지만, 가장 최근에 사용된 데이터가 다시 사용할 가능성을 고려하지 않는다. 예를 들어, A가 먼저 들어 오고 B가 그 이후에 들어왔다고 가정했을 때, A가 더 자주 사용된다면 A를 사용하여야 한다. 하지만, FIFO policy는 해당 부분에 상관 없이 A를 Replace 할 것이므로 Temporal Locality 면에서 떨어지는 결과를 가진다. 따라서, LRU Replacement Policy를 코드 설계에서 채택하였다. LRU Policy의 기본적인 아이디어는 가장 오래 사용 되지 않은 Cacheline을 교체 하는 것이다. 해당 LRU policy를 구현 하기 위해서는 Cacheline 에 변수 lru를 추가 해 주어야 한다. 변수 lru는 데이터가 접근 될 때 마다 해당 카운터 값을 업데이트(해당 lru 값을 0으로 바꾸어 주고 나머지는 더해주기)하며 교체할 데이터를 선택 할 때, 해당 변수 값이 가장 큰 값을 교체 하게 한다. 따라서, 교체 할 Cacheline을 고르는 알고리즘에는 Max 값을 선택해야 하는 알고리즘이 필수적으로 들어간다.

III. Cache 코드 설계

위에서 선택한 Cache 방식, 정책들을 종합하면 과제의 코드는 4-way Associative Mapped 방식, Write-Back write policy, LRU Replacement Policy를 따르는 캐시가 만들어진다. 이때, CacheLine의 구성요소는 tag, valid, dirty, lru, data 배열이 있다. 이 요소들을 개념적으로 나타내면 다음과 같다.

이를 구현한 C코드는 다음과 같다.



```

typedef struct {
    uint32_t tag;
    uint8_t data[CACHE_LINE_SIZE];
    int valid;
    int dirty;
    int lru;
    int accessed; // 출력용
} CacheLine;

typedef struct {
    CacheLine lines[CACHE_ASSOC];
} CacheSet;

typedef struct {
    CacheSet sets[CACHE_SET_SIZE];
} Cache;

Cache cache; // 전역 변수로 선언된 캐시
    
```

해당 코드에서 Cache 는 CacheSet의 모임이며 Cache Set size 는 1024 이다. CacheSet은 CacheLine 구조체 배열을 가지며 4-way Associative 이므로 CACHE_ASSOC 는 4값을 가진다.

Cache가 접근되는 순간은 다음 세 경우 이다.

1. Fetch

2. LW

3. SW

위 세 경우는 각각 Hit 와 Miss 할 때, 취하는 알고리즘이 다르다. 아래는 해당 상황에 Hit 와 Miss 했을 때의 각각의 알고리즘이다.

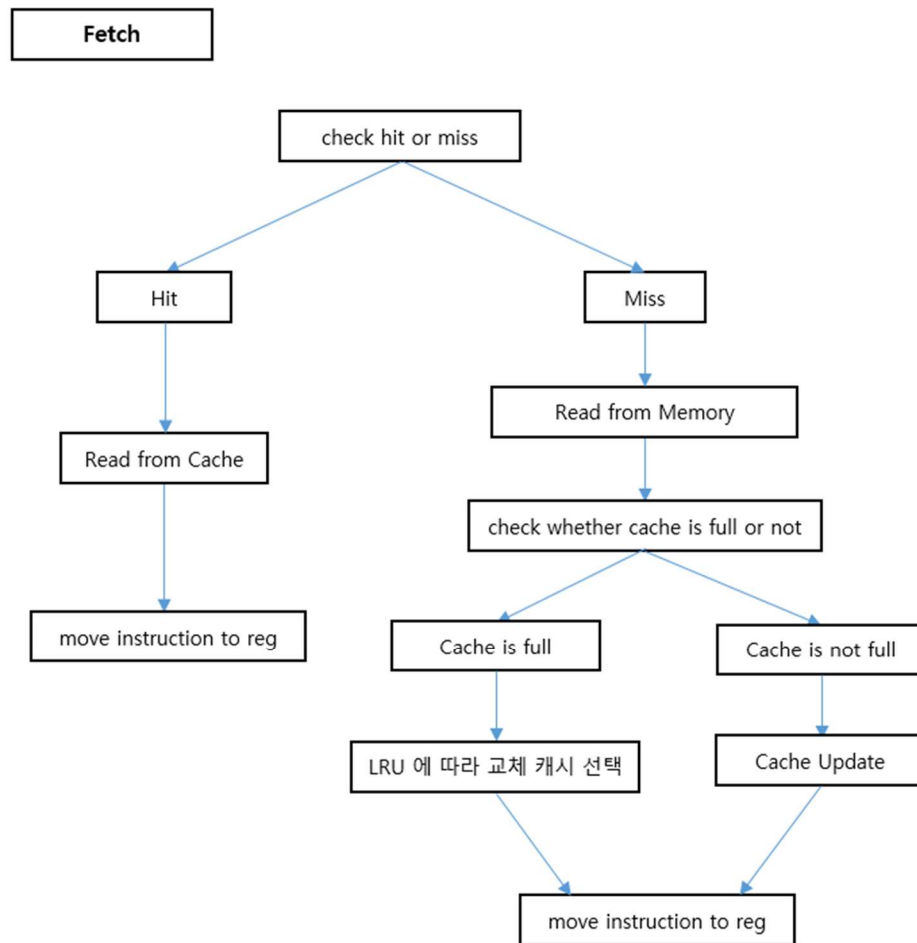
1 번의 Fetch 에서는 일반 데이터가 아닌 Instruction 을 다룬다. 이에 따라 LW 와 SW 와는 다르게 진행이 된다. 첫번째로, Cache 에서 명령어가 있는지 확인한다. 즉, Hit 인지 Miss 인지 확인한다.

만약, Hit 상황이 오면, Cache 에서 명령어를 읽어 오고 IFID Latch 에 읽어온 명령어를 저장한다. 이후, LRU 카운터를 업데이트 하여 가장 최근에 사용된 Cache Line 임을 표시한다. (밑에서 서술 및 표현 할 때 LRU 카운터는 생략된다.)

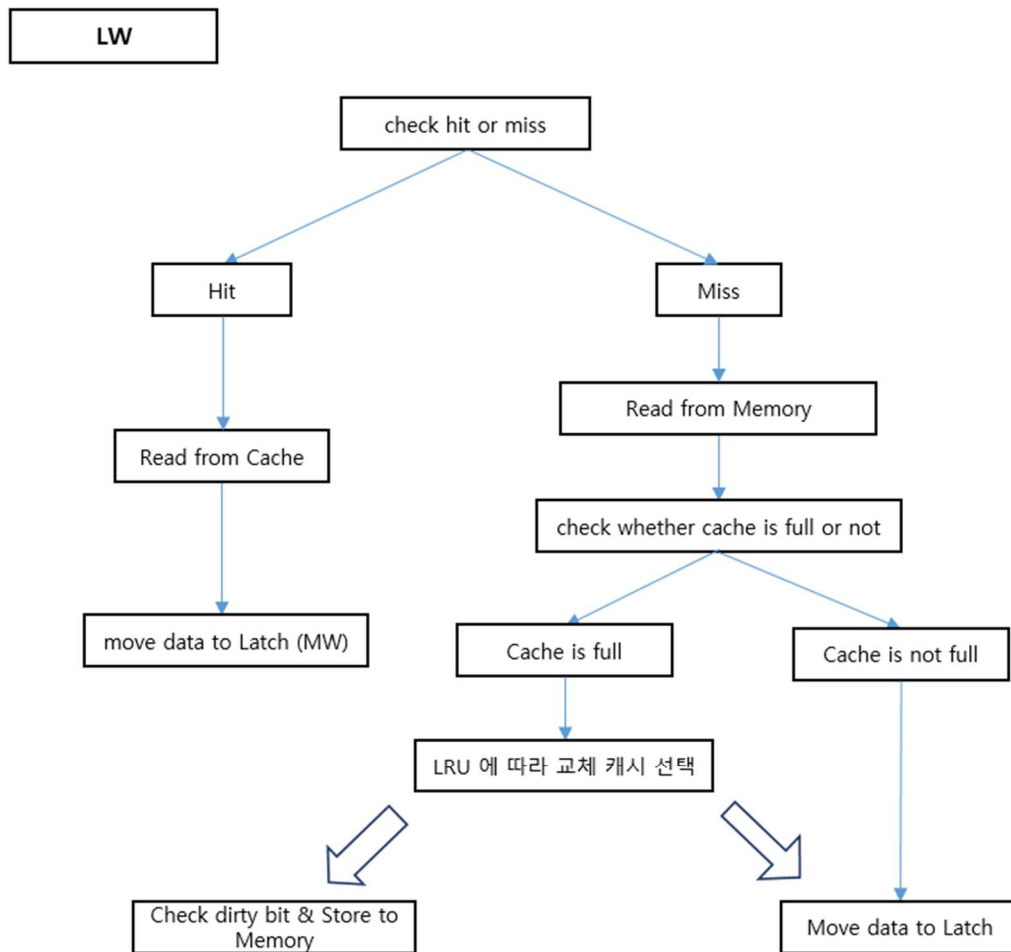
만약 Miss 상황이 오면, 해당 명령어를 Main memory 에서 읽어 온다. 이후, Cache Set 가 가득 찼는지 확인 하고, 모든 라인이 Valid 상태인 경우, LRU Policy 를 통해 교체할 캐시 라인을 선택 한다. 교체해야 할 CacheLine 의 경우, Dirty 값을 확인 한후, 해당 데이터를 Main Memory 에 저장한다. 만약 모든 라인이 Valid 상태가 아닌 경우, 메모리에서 가져온 명령어 값을 Cache 에 업데이트 한다. Cache 에 업데이트 할 때, 관련 태그를 업데이트 하며, 유효 비트를 설정한다.

이후 두 경우 모두 IFID Latch 에 Memory 에서 읽어온 명령어를 저장한다.

해당 상황에 대해 그림으로 표현하면 다음과 같다.



2 번의 LW 의 경우 Memory Access 단계에서 진행되며, 해당 명령어(LW)의 경우 Fetch 와 매우 유사하다. 왜냐하면, Fetch 에서 명령어를 읽기만 하기 때문에 LW 의 Cache Access 는 Fetch 와 공통점이 많다. 차이점은 LW 에 따라서 나온 데이터의 경우, LRU Policy 에 의해 교체될 때, dirty 값에 따라 Cache 값을 Store 해야 하지만, Fetch 는 가져온 주소에서 명령어를 바꾸면 안된다. LW 를 통해 Cache 나 Main memory 에서 온 값은 Memory Access 의 다음 단계인 Write Back 단계에서 사용되기 위해, MemWB Latch 로 이동 된다. 이를 그림으로 표현하면 다음과 같다.



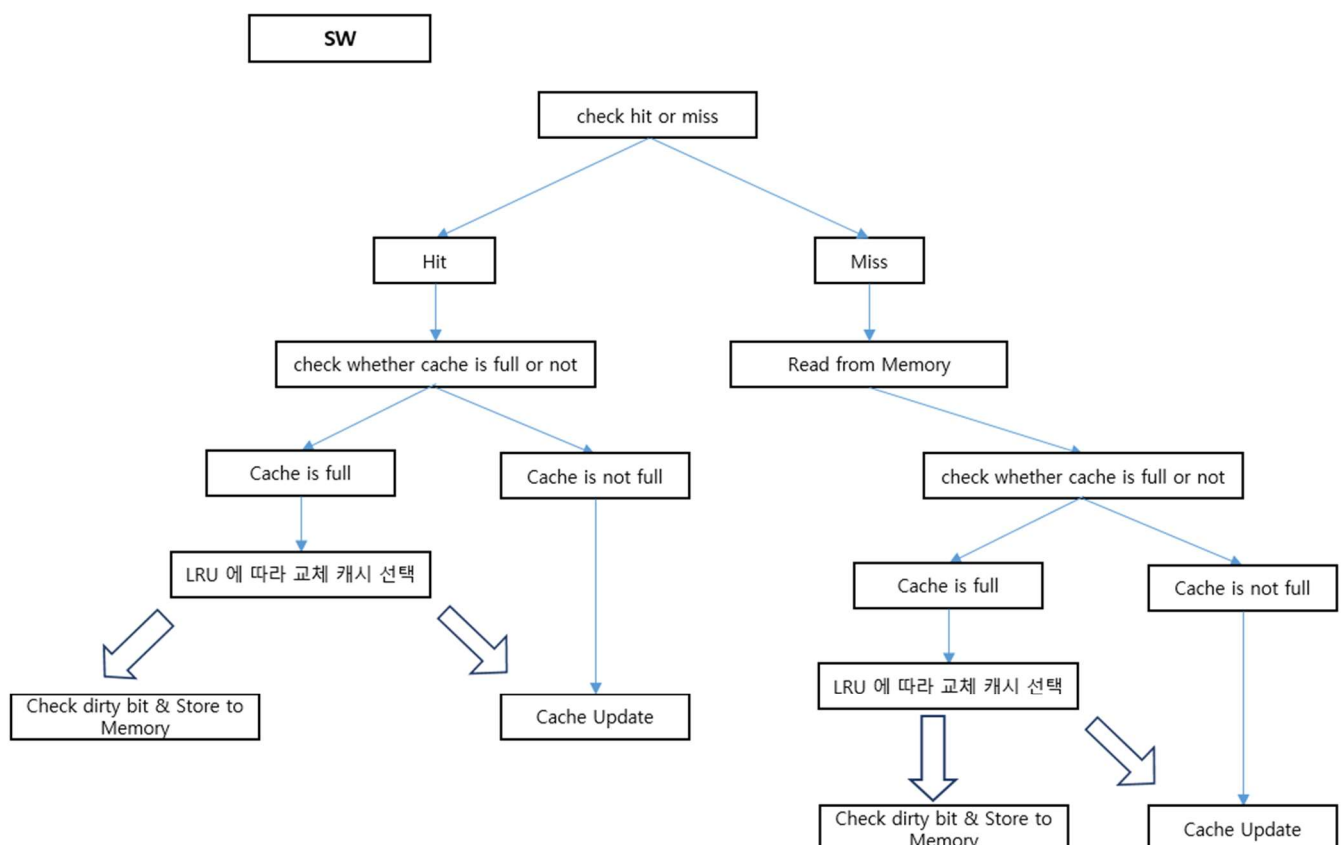
마지막으로 Store Word 의 경우 LW 와 다르게 복잡한 알고리즘을 가진다.

Cache Hit 인 경우, Cache Set 가 가득 차 있는지 확인한다.

만약, Cache Set 의 모든 Cache Line 이 유효한 상태라면, LRU policy 에 따라 교체할 Cache Line 을 선택한다. 이후, 선택된 Cache Line 에 데이터를 저장 한 후 교체 할 Cache Line 의 Dirty Bit 를 확인 한 후, Cache 값을 Main memory 에 Store 해준다. Cache 에 저장된 데이터의 경우 Dirty 값을 1 로 해주어야 한다. Cache set 의 모든 Cache set 유효하지 않다면, 가장 먼저 확인 된 곳에, Cache 값을 업데이트 해주기만 하면 된다.

Cache Miss 인 경우, Main memory 에서 해당 데이터를 읽어 온다. 이후, Cache set 가 비었는지 확인한다. 만약, Cache 를 업데이트 할 공간이 없다면, LRU policy 에 의해 처리하고, Main Memory 값을 dirty 값에 따라 수정해주면 된다. Replace 한 이후나 Replace 할 필요가 없다면, Main memory 에서 가져온 데이터를 선택된 Cache Line 에 업데이트하고 유효 비트를 설정한다. Cache 에 저장된 데이터에서 Dirty 값은 1 로 설정한다.

이에 대한 알고리즘을 그림으로 표현하면 다음과 같다.



위 코드에 대한 결과 값은 다음과 같다.

[simple]

```
=====
Return register (r2)           : 0
Total clock cycle              : 8
r-type count                   : 5
i-type count                   : 4
branch, j-type count, jr      : 3
lw count                      : 1
sw count                      : 1
register write count           : 5

cache access                   : 10
hit count                     : 1
cold miss                     : 9
conflict miss                 : 0
hit rate                      : 10.000 %
=====
```

[simple2]

```
=====
Return register (r2)           : 100
Total clock cycle              : 10
r-type count                   : 5
i-type count                   : 7
branch, j-type count, jr      : 3
lw count                      : 2
sw count                      : 2
register write count           : 7

cache access                   : 14
hit count                     : 2
cold miss                     : 12
conflict miss                 : 0
hit rate                      : 14.286 %
=====
```

[simple3]

```
=====
Return register (r2)           : 5050
Total clock cycle              : 1330
r-type count                   : 106
i-type count                   : 818
branch, j-type count, jr      : 106
lw count                       : 407
sw count                       : 206
register write count           : 715

cache access                   : 1943
hit count                      : 1914
cold miss                     : 29
conflict miss                  : 0
hit rate                       : 98.507 %
=====
```

[simple4]

```
=====
Return register (r2)           : 55
Total clock cycle              : 243
r-type count                   : 62
i-type count                   : 143
branch, j-type count, jr      : 34
lw count                       : 59
sw count                       : 41
register write count           : 151

cache access                   : 343
hit count                      : 264
cold miss                     : 79
conflict miss                  : 0
hit rate                       : 76.968 %
=====
```

[fib]

```
=====
Return register (r2)           : 55
Total clock cycle              : 2679
r-type count                   : 548
i-type count                   : 1588
branch, j-type count, jr      : 385
lw count                       : 601
sw count                       : 494
register write count           : 1530

cache access                   : 3774
hit count                      : 3679
cold miss                     : 95
conflict miss                  : 0
hit rate                       : 97.483 %
=====
```

[gcd]

```
=====
Return register (r2)           : 1
Total clock cycle              : 1061
r-type count                   : 224
i-type count                   : 564
branch, j-type count, jr      : 178
lw count                       : 333
sw count                       : 153
register write count           : 595

cache access                   : 1547
hit count                      : 1338
cold miss                     : 209
conflict miss                  : 0
hit rate                       : 86.490 %
=====
```

[input4]

```
=====
Return register (r2)           : 85
Total clock cycle              : 23372706
r-type count                   : 5076370
i-type count                   : 11190042
branch, j-type count, jr      : 2029805
lw count                       : 6090476
sw count                       : 1026130
register write count           : 15238464

cache access                   : 30489312
hit count                      : 29307100
cold miss                      : 4096
conflict miss                  : 1178116
hit rate                       : 96.123 %
=====
```

주어진 input4.bin 결과 값을 통해 Cache 성능에 대해서 생각해 보고자 한다.

메모리 지연이 없을 때, CPI 를 1 이라고 하고, 메모리 접근 Panalty 를 1000Cycle 이라고 가정하면

- (현재 Cache 사용 할 때 명령어 당 생기는 평균적인 메모리 지연)

$$\frac{(6090476 + 1026130)}{23372706} \times \frac{100 - 96.123}{100} \times 1000 \approx 11.8$$

- (완벽한 Cache 사용 할 때 명령어 당 생기는 평균적인 메모리 지연)

$$\frac{(6090476 + 1026130)}{23372706} \times \frac{100 - 100}{100} \times 1000 = 0$$

- (Cache 를 사용 하지 않을 때 명령어 당 생기는 평균적인 메모리 지연)

$$\frac{(6090476 + 1026130)}{23372706} \times \frac{100 - 0}{100} \times 1000 \approx 304.4$$

여기서 각각의 값에 1 을 더해주면 각각 12.8, 1, 305.4 가 나온다. 이 말인 즉, 완벽한 Cache 에 비해서 현재 Cache 를 사용하면 12.8 배 성능이 저하 된다는 뜻이며, Cache 를 사용하지 않으면 300 배 이상 성능이 저하된다는 의미이다. 현재 Simulator 의 Cache 를 사용하면 12.8 배 성능

저하가 생기는데, 이는 즉 Cache 의 Hit 율이 100%에 근접 할 수 록 성능이 10 배 이상 좋아 질 가능성이 있다는 뜻이다.

위의 여러 binary 파일들을 보면 작은 프로그램 (ex.simple3 까지) 들을 사용 할 때는 Cache 의 hit 율이 낮게 나온다. 이는 Cold miss 에 나오는 현상인데, Cache 를 프로그램의 초반부에 사용 할 때, 의무적인(반드시 나와야 하는) miss 가 있기 때문이다. 여러 결과 값은 프로그램의 크기가 커질수록 프로그램에서의 Cache 사용이 중요하다는 사실을 알 수 있게 만들어 준다. 즉, 프로그램이 커질수록 Cache hit 율은 어떠한 숫자에 수렴하게 된다.

마지막으로, Hit rate 을 높이기 위해 할 수 있는 것에 대해 생각해 보면 다음과 같다.

1. Cache 의 사이즈를 늘리는 것.

위 프로그램에서 Cache 사이즈를 늘리는 방법은 매우 간단하다. Cache 8 번 코드에 있는 CACHE_SET_SIZE 값을 2 의 제곱수에 맞게 늘려주면 된다. 아래는 CACHE_SET_SIZE 를 늘렸을때의 input4 결과들이다.

[CACHE_SET_SIZE = 64]

```
=====
Return register (r2)           : 85
Total clock cycle             : 23372706
r-type count                  : 5076370
i-type count                  : 11190042
branch, j-type count, jr     : 2029805
lw count                      : 6090476
sw count                      : 1026130
register write count          : 15238464

cache access                  : 30489312
hit count                     : 28109158
cold miss                     : 256
conflict miss                 : 2379898
hit rate                      : 92.193 %
=====
```


[CACHE_SET_SIZE = 128]

```
=====
Return register (r2)           : 85
Total clock cycle              : 23372706
r-type count                   : 5076370
i-type count                   : 11190042
branch, j-type count, jr      : 2029805
lw count                       : 6090476
sw count                       : 1026130
register write count           : 15238464

cache access                   : 30489312
hit count                      : 28649459
cold miss                     : 512
conflict miss                  : 1839341
hit rate                       : 93.966 %
=====
```

[CACHE_SET_SIZE = 256]

```
=====
Return register (r2)           : 85
Total clock cycle              : 23372706
r-type count                   : 5076370
i-type count                   : 11190042
branch, j-type count, jr      : 2029805
lw count                       : 6090476
sw count                       : 1026130
register write count           : 15238464

cache access                   : 30489312
hit count                      : 29035832
cold miss                     : 1024
conflict miss                  : 1452456
hit rate                       : 95.233 %
=====
```

[CACHE_SET_SIZE = 2048]

```
=====
Return register (r2)           : 85
Total clock cycle              : 23372706
r-type count                   : 5076370
i-type count                   : 11190042
branch, j-type count, jr      : 2029805
lw count                       : 6090476
sw count                       : 1026130
register write count           : 15238464

cache access                   : 30489312
hit count                      : 29320374
cold miss                     : 8192
conflict miss                  : 1160746
hit rate                       : 96.166 %
=====
```

위 그림들의 Conflict miss 값과 Hit Rate 에 주목해 보면 Cache Size 가 커질수록 Conflict miss 가 줄어드는 것을 볼 수 있다. 당연한 결과 이지만, Cache 사이즈가 커지면 Hit Rate 이 높아진다.

2. 여러 Cache 를 사용하는 것(미구현)

Miss 가 나오는 과정의 로그를 살펴본 결과 Fetch 와 Memory Access 의 Miss 율이 다르다는 사실을 알게 되었다. 특히, Fetch 의 Miss 율이 높았는데, 이에 따라, Fetch 용 Cache 와 LW SW 용 Cache 를 나누는 것 즉, Cache 를 여러 개 사용하면 성능이 높을 것이라는 추측이 든다.

결론

이번 과제를 통해 Cache 에 대한 이해를 높일 수 있었다. Cache 의 성능 비교를 통해 알 수 있듯이, Cache 기술은 컴퓨터 구조의 성능을 올리는데 필수 불가결한 요소 중 하나이다. 컴퓨터 구조의 기술의 발전에 대해서 여러 과제를 통해 알 수 있었는데, 컴퓨터 구조에서 문제점이 생긴다면 이에 대한 해결책을 찾고 해결책으로 부족한다면 새로운 아이디어와 기술들을 도입하여 해당 문제점을 해결하고 발전하였다. 이와 같이, 이번 학기의 컴퓨터 구조 수업을 통해 과제에 부딪히며 나의 코딩 실력과 컴퓨터에 대한 시야가 넓어졌다고 생각한다.

빌드 방식 : VS 2019

로그 없이 결과값 보는 방법 : printf 부분 들을 모두 //printf 로 바꾸어준후 1394~1408 번째 줄은 주석을 풀어주시면 됩니다.

고생하셨습니다!